Introduction to Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI **4**
1.1 Rational decisions . . . 4
1.2 Agents and environments . . . 4
1.3 Methods and applications . . . 5

2 Uninformed Search **6**
2.1 Planning and search problems . . . 6
2.2 State graphs and search trees . . . 6
2.3 Depth-first and breadth-first search . . . 7
2.4 Uniform-cost search . . . 7

3 A* Search and Heuristics **8**
3.1 Informed search . . . 8
3.2 Constructing heuristics . . . 8
3.3 Graph search and consistency . . . 8

4 Constraint Satisfaction Problems I **10**
4.1 Representation and backtracking . . . 10
4.2 Filtering and arc consistency . . . 10
4.3 Ordering . . . 11

5 Constraint Satisfaction Problems II **12**
5.1 Consistency and structure . . . 12
5.2 Local search . . . 13

6 Game Trees: Minimax **14**
6.1 Games and minimax values . . . 14
6.2 Alpha-beta pruning . . . 14
6.3 Depth limits and evaluation . . . 15

7 Game Trees: Expectimax and Utilities **16**
7.1 Chance nodes and opponent models . . . 16
7.2 Preferences over lotteries . . . 16
7.3 Money and risk . . . 17

8 Markov Decision Processes I **18**
8.1 States, rewards, and policies . . . 18
8.2 Bellman optimality and time-limited values . . . 19

9 Markov Decision Processes II **20**
9.1 Policy evaluation and extraction . . . 20
9.2 Policy iteration . . . 21
9.3 From planning to learning . . . 21

10 Reinforcement Learning I **22**
10.1 Learning a model or learning values . . . 22
10.2 Temporal differences . . . 22
10.3 Q-learning . . . 23

11 Reinforcement Learning II **24**
11.1 Exploration and regret . . . 24
11.2 Approximate Q-learning . . . 25
11.3 Policy search . . . 25

12 Probability **26**
12.1 Distributions, events, and conditioning . . . 26
12.2 Product, chain, and Bayes rules . . . 27
12.3 Inference by enumeration . . . 27

13 Bayes Nets: Representation **28**
13.1 Conditional independence . . . 28
13.2 Graph and local probabilities . . . 28
13.3 Representation and causality . . . 29

14 Bayes Nets: Independence **30**
14.1 Three local configurations . . . 30
14.2 D-separation . . . 31
14.3 Modeling consequences . . . 31

15 Bayes Nets: Inference **32**
15.1 Factors and elimination . . . 32
15.2 Evidence and normalization . . . 32
15.3 Ordering and complexity . . . 33

16 Bayes Nets: Sampling **34**
16.1 Prior and rejection sampling . . . 34
16.2 Likelihood weighting . . . 34
16.3 Gibbs sampling . . . 35

17 Decision Networks and Value of Information **36**
17.1 Decisions and expected utility . . . 36
17.2 Value of information . . . 37
17.3 Partial observability . . . 37

18 Hidden Markov Models **38**
18.1 Markov chains and stationary distributions . . . 38
18.2 Hidden states and observations . . . 38
18.3 Filtering and the forward algorithm . . . 39

19 Particle Filtering and Applications of HMMs **40**
19.1 Particles as a belief representation . . . 40
19.2 Dynamic Bayes nets . . . 40
19.3 Most likely state sequences . . . 41

20 Machine Learning: Naive Bayes **42**
20.1 Classification and the model . . . 42
20.2 Parameter estimation and smoothing . . . 42
20.3 Generalization and evaluation . . . 43

21 Machine Learning: Perceptrons and Logistic Regression **44**
21.1 Linear scores and perceptron updates . . . 44
21.2 Logistic probabilities . . . 45
21.3 Multiclass logistic regression . . . 45

22 Machine Learning: Optimization and Neural Networks **46**
22.1 Gradient optimization . . . 46
22.2 Learning representations . . . 47
22.3 Approximation and automatic differentiation . . . 47

23 Machine Learning: Neural Networks II and Decision Trees **48**
23.1 Induction, consistency, and simplicity . . . 48
23.2 Decision trees and expressiveness . . . 48
23.3 Information gain and recursive learning . . . 49
23.4 Overfitting and pruning . . . 49

24 Advanced Applications: Robotics, Vision, and Control **50**
24.1 Combining search and learning . . . 50
24.2 Helicopter estimation and control . . . 50
24.3 Locomotion and rewards . . . 51
24.4 Autonomous driving and manipulation . . . 51

25 Advanced Topics and Final Contest **52**
25.1 Cooperative game strategies . . . 52
25.2 Speech as sequential inference . . . 53
25.3 Connecting inference to action . . . 53

Lecture 1: Introduction to AI

1.1 Rational decisions

Artificial intelligence can be approached through four goals: making machines think like people, act like people, think rationally, or act rationally. This course focuses on the last: choosing actions that best achieve specified goals. A decision can be rational even if it does not resemble human thinking.

A rational decision maximizes expected utility, a numerical measure of the desirability of an outcome. If h denotes the information available when choosing an action, O is the set of possible outcomes, and $U(o)$ is the utility of outcome o, then for a finite outcome set the rule is

$$a^* \in \arg\max_a \sum_{o \in O} P(o \mid h, a)\, U(o).$$

For each action, weight each possible outcome's utility by its probability, then choose an action with the largest total. Rationality is relative to the goals and information available at decision time: an unlikely bad outcome can occur even when the chosen action had the highest expected utility. The criterion evaluates the decision, not whether its internal reasoning imitates a person.

1.2 Agents and environments

An agent perceives its environment and acts on it. Sensors deliver percepts, and actuators carry out actions. The environment then changes and supplies new percepts, so decisions form a continuing interaction. The nature of the observations, available actions, and environment determines which techniques are appropriate.

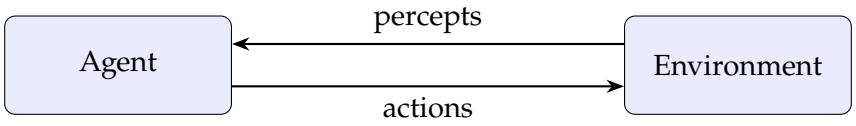


Figure 1: An agent interacts with its environment.

For Pac-Man, the environment is the game, percepts describe what the agent can observe, and actions move Pac-Man through the maze. The agent's objective specifies what counts as a good outcome; seeing the same game does not by itself determine what the agent should prefer.

A REFLEX AGENT chooses from its current percept, possibly using memory, without explicitly evaluating future consequences. A PLANNING AGENT uses a model to consider how candidate actions might change the world and selects a plan in light of its goal. A reflex rule can still be rational when it selects an optimal action for the setting; planning is one way to make decisions, not the definition of rationality.

1.3 Methods and applications

AI systems combine computation and data. Search and planning compare possible actions; constraint satisfaction finds assignments that obey specified rules; adversarial and uncertain settings require reasoning about other agents or uncertain outcomes. Probabilistic models represent uncertainty, decision theory connects uncertainty to utility, and machine learning uses data to improve predictions or decisions.

The same principles appear in different applications:

- **Language:** speech recognition maps audio to text, speech synthesis maps text to audio, and language systems support translation, question answering, and text classification.
- **Vision:** perception turns pixels into information, such as object identities, scene labels, or three-dimensional structure.
- **Robotics:** perception, planning, and control connect decisions to physical actions. The real world can differ from a simulator.
- **Games and logic:** game-playing systems reason over moves; logical systems use deduction, constraint satisfaction, and satisfiability to solve structured problems.

The field has used both knowledge-based and statistical approaches. Recent systems in the 2018 course draw on large datasets, computation, and neural networks; the appropriate representation and objective still depend on the task.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 1 and Note 1.

Lecture 2: Uninformed Search

2.1 Planning and search problems

A planning agent considers possible consequences before acting. It needs a model of how actions change the world and a goal test. A plan can be computed in advance or recomputed as new observations arrive. Search evaluates possible plans in the model, so its conclusions depend on the model's accuracy.

A search problem specifies a state space, a start state, a goal test, and a successor function. For a state, the successor function gives the available actions, resulting states, and step costs. A solution is a sequence of actions taking the start state to a goal; its cost is the sum of step costs. An optimal solution has minimum cost among all solutions.

A search state is an abstraction of the world: it retains the information needed to determine successors, costs, and goal status. For pathfinding in a maze, a position may suffice. To eat every dot, the state must also record which dots remain. If there are n possible positions and k dots, this representation has at most $n2^k$ states. Adding irrelevant details enlarges the search; omitting a detail that affects future actions or the goal makes the model inadequate.

2.2 State graphs and search trees

In a state-space graph, each state is represented once, and an edge represents an action and its successor. A search tree unfolds the possible plans from the start: its root is the start and each child extends its parent's plan by one action. A tree node represents a path as well as its end state. Thus different nodes can end at the same state, and a cycle in a finite state graph can generate an infinite search tree. Both structures are usually constructed only as needed.

Tree search maintains a fringe of partial plans. It removes a node according to an exploration strategy, checks whether its state is a goal, and otherwise expands it by adding successor plans. The strategy determines which plans are considered first. A goal is returned when selected from the fringe, which is especially important when path costs differ.

We compare strategies by completeness, whether a solution is found when one exists, and optimality, whether the returned solution has least cost. Let b be the maximum branching factor, m the maximum tree depth, and s the depth of the shallowest goal. These tree-search bounds count nodes, including repeated states; assume finite b.

2.3 Depth-first and breadth-first search

Depth-first search (DFS) expands a deepest fringe node first, using a last-in, first-out stack. It can pursue one branch indefinitely, so tree search is not complete for an infinite-depth tree. With finite depth it takes $O(b^m)$ time in the worst case. Its fringe holds at most the unexplored siblings along a root-to-leaf path, requiring $O(bm)$ space. DFS need not find a shallowest or least-cost solution.

Breadth-first search (BFS) expands a shallowest fringe node first, using a first-in, first-out queue. It explores by depth and is complete when branching is finite and a solution exists at finite depth. It finds a shallowest goal, which is optimal when all step costs are the same positive constant. Its time and space are exponential in solution depth; a conservative bound for the usual goal test on removal is $O(b^{s+1})$, because expansions at depth s before the goal can generate depth $s + 1$ nodes.

Iterative deepening runs depth-limited DFS repeatedly with increasing limits. It finds a shallowest goal with the space advantage of DFS. Although it revisits upper levels, for $b > 1$ most nodes lie near the deepest level, so the repeated work changes the asymptotic time by only a constant factor.

2.4 Uniform-cost search

When step costs differ, the fewest actions need not yield the cheapest plan. Uniform-cost search (UCS) uses a priority queue ordered by the cumulative path cost g. It removes a cheapest partial plan, tests for a goal, and expands it if necessary. It explores in increasing cost contours, without using information about the direction of the goal.

Suppose branching is finite, every step costs at least $\epsilon > 0$, and an optimal goal has finite cost C^*. UCS is complete: only finitely many tree nodes can have cost at most C^*, since their depth is at most C^*/ϵ. It is also optimal. If a more expensive goal were removed first, a frontier prefix of an optimal path would have cost at most C^* and would have been selected before that goal. This argument relies on nonnegative step costs and on checking the goal when it is removed, not merely when it is generated. Time and space are exponential in the effective depth C^*/ϵ, with implementation-dependent boundary factors from generating the next level.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 2.

Lecture 3: A* Search and Heuristics

3.1 Informed search

A heuristic $h(s)$ estimates the remaining cost from state s to a nearest goal. Greedy search expands the node with smallest h, ignoring the cost already paid. It can be fast when the estimate points toward a goal, but can follow a misleading route and need not find a least-cost solution.

A* search instead orders the fringe by $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of the path represented by node n. With $h = 0$ it is uniform-cost search. A goal is tested when removed from the fringe: the first goal generated need not be the cheapest.

A heuristic is admissible if $0 \le h(s) \le h^*(s)$, where h^* is the true least remaining cost. In particular, h is zero at goals. For finite branching and step costs bounded below by a positive constant, A* tree search with an admissible heuristic is complete and optimal. To see optimality, suppose a suboptimal goal of cost $C > C^*$ is on the fringe. Some prefix n of an optimal path is also on the fringe until the optimal goal is removed. Admissibility gives $f(n) \le C^* < C = f(\text{goal})$, so the suboptimal goal cannot be removed first.

3.2 Constructing heuristics

A relaxed problem removes restrictions on actions; its optimal cost is a lower bound on the original cost and therefore an admissible heuristic. In the sliding-tile puzzle, the number of misplaced tiles is a lower bound on moves. The sum of the tiles' Manhattan distances to their goal positions is another: each move changes one tile's distance by at most one. It corresponds to relaxing the restrictions imposed by the other tiles and the blank.

If $h_1(s) \ge h_2(s)$ everywhere, h_1 dominates h_2. A more informative admissible estimate generally avoids expanding nodes with $f > C^*$, though ties and computation cost matter. The maximum of admissible heuristics remains admissible. The exact remaining cost is ideal as a bound, but computing it can be as hard as the original problem.

3.3 Graph search and consistency

Tree search may revisit a state through many paths. Graph search keeps a closed set of expanded states and skips later occurrences. With costs, closing a state reached by a poor path can discard a better path discovered later; admissibility alone does not prevent this in A*.

A heuristic is CONSISTENT if for every edge $s \to s'$ of cost c,

$$h(s) \leq c(s, s') + h(s'), \qquad h(g) = 0 \text{ at goals.}$$

This triangle inequality implies f cannot decrease along a path. It also implies admissibility by summing along a path to a goal. With a consistent heuristic, the first expansion of a state is along a cheapest path to it: a frontier prefix of any cheaper path would have no greater f than that cheaper arrival, and hence would be expanded first. Consequently A* graph search that closes states on expansion is optimal. An alternative for an admissible but inconsistent heuristic is to reopen a state when a cheaper path is found.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 3.

Lecture 4: Constraint Satisfaction Problems I

4.1 Representation and backtracking

A constraint satisfaction problem (CSP) consists of variables $X_1, \ldots, X_n$, domains D_i of possible values, and constraints on allowed combinations of values. A solution assigns every variable a value and satisfies all constraints. Unlike path planning, the assignment itself is the goal. A constraint may be unary, binary, or involve more variables; it can be stated as a rule or a list of allowed tuples.

For map coloring, variables are regions, domains are colors, and adjacent regions must differ. In a binary CSP the constraint graph has a vertex for each variable and an edge for each pair constrained together. Disconnected components are independent subproblems. With n variables and at most d values each, there can be d^n complete assignments.

Backtracking search performs depth-first search over partial assignments. At each step it picks one unassigned variable and tries its values, rejecting an assignment as soon as it violates a constraint. Assignments commute, so it is unnecessary to branch over all possible orders in which the same variables could be assigned. If no value works, undo the most recent choice and try another. Filtering and ordering can reduce the search without changing its set of solutions.

4.2 Filtering and arc consistency

Forward checking removes from each unassigned neighbor's domain values inconsistent with a new assignment. An empty domain proves failure, but forward checking does not detect every conflict between unassigned variables.

A directed arc $X \to Y$ is arc consistent if every value in D_X has a supporting value in D_Y satisfying the constraint between them. To revise an arc, delete unsupported values from the *tail* domain. To propagate constraints, repeatedly revise arcs; whenever D_X shrinks, recheck arcs from neighbors into X. An empty domain proves that the current branch has no solution. Domains must be restored on backtracking. Arc consistency can be enforced before search and after each assignment.

Arc consistency is not sufficient for global satisfiability. For example, three pairwise unequal variables forming a triangle, each with domain $\{\text{red}, \text{blue}\}$, are arc consistent but cannot all be assigned. Propagation therefore complements rather than replaces search. A basic implementation takes $O(n^2 d^3)$ time in the worst case; more efficient support tracking can improve this bound.

4.3 Ordering

The MINIMUM REMAINING VALUES rule chooses an unassigned variable with the smallest current domain, exposing failure early. The LEAST CONSTRAINING VALUE rule tries the value that eliminates the fewest choices from other variables. The first rule chooses where to branch; the second chooses which branch to try first. Evaluating these choices and propagating constraints has a cost, traded against the search it saves.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 4.

Lecture 5: Constraint Satisfaction Problems II

5.1 Consistency and structure

A CSP is k-consistent if any consistent assignment to $k-1$ variables can be extended to any kth variable. It is strongly k-consistent if it is also j-consistent for every $1 \leq j < k$. Strong n-consistency permits a solution to be built one variable at a time without backtracking, but enforcing higher-order consistency can be expensive.

The constraint graph can make a problem easier. Separate connected components can be solved independently. If n variables split into components of at most c variables, each with domain size at most d, exhaustive search costs roughly $O((n/c)d^c)$ for equal-sized components rather than $O(d^n)$.

A binary CSP whose constraint graph is a tree can be solved in $O(nd^2)$. Choose a root and order parents before children. Process children in reverse order, making each parent-to-child arc consistent by deleting unsupported parent values. Then assign the root and proceed forward, choosing for each child a value compatible with its parent. The backward pass guarantees support for each surviving parent value; once a child has been processed, later revisions do not remove its support. Thus the forward pass needs no backtracking. An empty domain signals failure.

For a nearly tree-structured graph, choose a cutset whose removal leaves a tree or forest. Try each assignment to the cutset, reject inconsistent ones, restrict the remaining domains, and solve the resulting forest. A cutset of size c gives a bound of order $d^c(n-c)d^2$, apart from checking the cutset constraints. Tree decompositions use overlapping groups of variables arranged in a tree, with agreement on shared variables.

5.2 Local search

Local search keeps a single complete assignment and changes it rather than maintaining a fringe of partial assignments. For a CSP, MIN-CONFLICTS selects a variable involved in a violated constraint and assigns it a value that minimizes the number of conflicts. It can be effective on large problems, but can become stuck or cycle and is not generally complete.

More generally, HILL CLIMBING repeatedly moves to a better neighbor and stops when none is better. Local optima and plateaus can prevent it from reaching a global optimum. SIMULATED ANNEALING sometimes accepts a worse move: for objective decrease $\Delta < 0$, a common acceptance probability is $e^{\Delta/T}$. As temperature T falls, downhill moves become less likely. Slow cooling has theoretical convergence guarantees, but escape can still be slow. Genetic algorithms select a population by fitness and create candidates using crossover and mutation. Their usefulness depends on the representation.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 5.

Lecture 6: Game Trees: Minimax

6.1 Games and minimax values

A deterministic game specifies states, legal actions, a transition function, whose turn it is, a terminal test, and terminal utilities for each player. A solution is a policy that chooses an action in each state. In a two-player zero-sum game, one player's gain is the other's loss. Write all values from MAX's perspective; MIN minimizes that same value.

The minimax value is the best utility MAX can guarantee against an optimal adversary. For successors $\mathrm{Succ}(s)$,

$$V(s) = \begin{cases} U(s), & s \text{ terminal}, \\ \max_{s' \in \mathrm{Succ}(s)} V(s'), & \text{MAX's turn}, \\ \min_{s' \in \mathrm{Succ}(s)} V(s'), & \text{MIN's turn}. \end{cases}$$

Choose a successor attaining the maximum at MAX's root. The recursion follows from each player choosing its best continuation. A minimax strategy is safe against optimal opposition, but can be excessively cautious against an opponent whose errors could be exploited. For branching factor b and depth m, full minimax takes $O(b^m)$ time and $O(bm)$ space with depth-first traversal.

6.2 Alpha-beta pruning

Alpha-beta pruning avoids branches that cannot affect the root's minimax decision. Carry α, the best alternative already available to MAX along the current path, and β, the best alternative available to MIN. Initialize them to $-\infty$ and $+\infty$.

At a MAX node, accumulate the largest child value, update α, and stop if the running value is at least β: an ancestor MIN already has an option no larger than β. At a MIN node, accumulate the smallest child value, update β, and stop if it is at most α. Unexamined children cannot change the ancestor's choice. A returned value after a cutoff can be only a bound on that internal node's true value; it should not be treated as an exact evaluation of every root child.

Pruning preserves the root value. Its effectiveness depends on ordering: examine promising moves first so useful bounds become available early. The worst case remains $O(b^m)$; ideal ordering reduces the leaf count to order $b^{m/2}$, permitting roughly twice the depth for the same budget.

6.3 Depth limits and evaluation

Realistic trees are too large to reach terminal states. At a depth limit, replace terminal utility by an EVALUATION FUNCTION, often $\text{Eval}(s) = \sum_i w_i f_i(s)$ for features such as material advantage or distance to food. This removes the optimal-play guarantee: search now optimizes the truncated model and its estimates.

Greater depth can reduce sensitivity to evaluation error, but expensive features leave less time to search. Iterative deepening provides a completed shallower result while deeper search proceeds. Evaluation also guides move ordering for alpha-beta. Replanning with a short horizon can postpone a good action forever when taking it now and taking it just beyond the horizon look equally valuable; the evaluation must distinguish useful progress from delay.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 6.

Lecture 7: Game Trees: Expectimax and Utilities

7.1 Chance nodes and opponent models

Uncertain outcomes may come from random events, action failures, or an opponent whose behavior is modeled probabilistically. Expectimax uses MAX nodes for choices and chance nodes for outcomes:

$$V(s) = \max_a V(s,a), \qquad V(s,a) = \sum_{s'} P(s' \mid s,a)V(s').$$

At terminal nodes use utility; at a depth cutoff use an estimate of future expected utility. For example, outcomes $8, 24, -12$ with probabilities $1/2, 1/3, 1/6$ have expectation $4 + 8 - 2 = 10$.

The probabilities encode a model, even when the opponent does not literally randomize. A model of an opponent that usually chooses a minimax move can put most mass on that move and the remainder on alternatives. Computing such a model may itself require search. A wrong model can be costly: optimism about an adversarial opponent can be dangerous, while worst-case reasoning about a random opponent can miss worthwhile opportunities.

An unexplored chance outcome can have an arbitrarily large value, so ordinary alpha-beta cutoffs do not apply without bounds on utilities. Games with both opposition and chance use MAX, MIN, and chance layers as appropriate. With multiple independent utilities, a node can carry a utility vector and the player to move chooses the child maximizing its own component, with a specified rule for ties.

7.2 Preferences over lotteries

A lottery is a probability distribution over outcomes. Rational preferences satisfy constraints including completeness, transitivity, continuity, substitutability, monotonicity in the probability of a preferred prize, and reduction of compound lotteries to their outcome probabilities. Under the expected-utility representation, an outcome utility U can be chosen so that a lottery L is valued by

$$U(L) = \sum_o P_L(o)U(o).$$

The maximum expected utility principle chooses the action with greatest expected utility given the available information. A rational policy need not explicitly calculate this expression; its decisions can implement the same preferences by another mechanism.

Minimax decisions are unchanged by any strictly increasing transformation of terminal utilities because only order matters. Expectimax depends on the relative spacings: only positive affine transformations $U' = aU + b$, $a > 0$, preserve all lottery preferences. Normalize best and worst outcomes to 1 and 0; an outcome indifferent to a lottery paying the best outcome with probability p then has utility p.

7.3 Money and risk

Expected money and expected utility are different objectives. A concave utility of wealth expresses risk aversion: the utility of a sure mean can exceed the mean utility of a lottery. A CERTAINTY EQUIVALENT is the sure amount judged equivalent to a lottery; its difference from expected money measures the amount the agent would pay to remove the risk. Human preferences need not satisfy the axioms. In the Allais example, the common choices \$3000 for sure over an 80% chance of \$4000, and a 20% chance of \$4000 over a 25% chance of \$3000, imply contradictory inequalities for any single expected-utility function with $U(0) = 0$.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 7.

Lecture 8: Markov Decision Processes I

8.1 States, rewards, and policies

A Markov decision process (MDP) specifies states S, actions $A(s)$, transitions $T(s, a, s') = P(s' \mid s, a)$, rewards $R(s, a, s')$, a start state, and possibly terminal states. The Markov assumption says that the current state and action determine the distribution of the next state, without additional dependence on history.

In a noisy grid world, a requested move can go in the intended direction with probability 0.8 and each perpendicular direction with probability 0.1; a blocked move leaves the state unchanged. A living reward and terminal rewards express preferences between speed, safety, and remaining in the grid. The solution is a policy $\pi : S \to A$ rather than a fixed sequence: the action must respond to the state actually reached.

For discount $0 \le \gamma < 1$, the return from rewards $r_0, r_1, \ldots$ is $G = \sum_{t \ge 0} \gamma^t r_t$. With bounded rewards $|r_t| \le R_{\max}$, $|G| \le R_{\max}/(1 - \gamma)$. Discounting favors earlier rewards and makes infinite-horizon values finite. Finite horizons instead end after a fixed number of steps, and the best action can depend on time remaining. Undiscounted episodic problems need conditions ensuring termination and finite returns.

8.2 Bellman optimality and time-limited values

Let $V^*(s)$ be the optimal expected return starting at s, and $Q^*(s,a)$ the expected return after committing to action a and then acting optimally. Decomposing the return into the next reward and the discounted remainder gives

$$Q^*(s,a) = \sum_{s'} T(s,a,s')\big[R(s,a,s') + \gamma V^*(s')\big], \qquad V^*(s) = \max_a Q^*(s,a).$$

Terminal continuation values are zero when their rewards have already been paid on entry. These equations reuse values of repeated states instead of unfolding a separate infinite expectimax tree for every path.

Value iteration begins with $V_0(s) = 0$ and computes

$$V_{k+1}(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')].$$

Here V_k is the best value with k steps remaining. Use the previous vector throughout a synchronous iteration. A dense update costs $O(|S|^2|A|)$; sparse transition models require summing only over possible successors.

For a finite-depth problem, enough iterations recover the full value. For bounded rewards and $\gamma < 1$, the tail beyond depth k has magnitude at most $\gamma^k R_{\max}/(1-\gamma)$. Thus the truncation error vanishes and V_k converges to V^*. The maximizing actions can stabilize much earlier than the numerical values.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 8.

Lecture 9: Markov Decision Processes II

9.1 Policy evaluation and extraction

For a fixed deterministic policy π, the policy value is $V^\pi(s) = \mathbb{E}_\pi[\sum_{t\geq 0} \gamma^t r_t \mid s_0 = s]$. Fixing the action removes the maximum from the Bellman equation:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Compute it by iterating this update, at $O(|S|^2)$ per dense pass, or by solving the linear system $(I - \gamma T_\pi)V^\pi = r_\pi$. For a finite MDP with $\gamma < 1$, the system has a unique solution.

Policy extraction from a value estimate requires one-step lookahead:

$$\pi_V(s) \in \arg\max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V(s')].$$

Choosing the successor with largest value alone ignores rewards and transition probabilities. From optimal action values, extraction is simply $\pi^*(s) \in \arg\max_a Q^*(s, a)$.

9.2 Policy iteration

Policy iteration alternates two operations: evaluate the current policy to obtain V^π, then replace it by the greedy policy π_{V^π}. The improvement step cannot lower value: it chooses a first action at least as good as the old policy's action, followed by continuation values under the old policy. Repeated application of the improved policy preserves that inequality. If no action changes, the values satisfy Bellman optimality and the policy is optimal. Retain the old action on ties to avoid cycling between equally good policies.

Value iteration updates values and implicitly reselects every action each pass. Policy iteration spends several cheaper passes evaluating one fixed policy before reconsidering actions. Both solve the same known MDP; the computational advantage depends on the problem and evaluation method.

9.3 From planning to learning

Offline planning computes values and policies from known transitions and rewards without interacting with the environment. When those quantities are unknown, an agent must act and observe. Repeated outcomes estimate their probabilities, but learning incurs mistakes. Exploration gathers information about uncertain actions; exploitation uses current knowledge for reward. Learning a good final policy and collecting good rewards during learning are different objectives.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 9.

Lecture 10: Reinforcement Learning I

10.1 Learning a model or learning values

Reinforcement learning retains the MDP objective but learns from observed transitions (s, a, r, s') instead of being given T and R. Model-based learning counts successor outcomes to estimate $\widehat{T}(s, a, s') = N(s, a, s')/N(s, a)$ and estimates rewards from observations. It then solves the estimated MDP. Unvisited state-action pairs need further experience before their model can be trusted.

In passive learning, the policy is fixed and the task is evaluation. Direct evaluation averages observed discounted returns following visits to each state. It is model free and simple, but learns states separately rather than exploiting known estimates of their successors; it also waits for a return to become available.

10.2 Temporal differences

Temporal-difference learning updates on each transition under the fixed policy:

$$\delta = r + \gamma V(s') - V(s), \qquad V(s) \leftarrow V(s) + \alpha\delta.$$

The target is one observed reward plus an estimated continuation. Sampling outcomes automatically weights them by their transition probabilities in expectation. This bootstrapping shares information through the connections between states without estimating a transition table.

The update is an interpolation $(1 - \alpha)V(s) + \alpha[r + \gamma V(s')]$. A constant learning rate emphasizes recent samples; suitably decreasing rates allow averages to settle. Terminal continuation values are zero. TD evaluation alone does not supply a model-free improvement rule from state values: deciding among actions would still require their transition and reward models.

10.3 Q-learning

Active learning chooses actions as well as evaluating them. The model-based Bellman update for action values is

$$Q_{k+1}(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')].$$

Q-learning replaces this expectation by sampled updates:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)].$$

Action selection then uses $\arg\max_a Q(s,a)$ without a transition model. Q-learning is off-policy: the greedy continuation in its target can differ from the exploratory behavior that collected the data. In the finite, discounted tabular setting with bounded rewards, sufficient repeated visits to every state-action pair and per-pair rates satisfying $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$ permit convergence to Q^*. Merely acting greedily from the start need not provide that coverage.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 10.

Lecture 11: Reinforcement Learning II

11.1 Exploration and regret

An ϵ-greedy behavior policy selects a random action with probability ϵ and a currently greedy action otherwise. Persistent randomness provides exploration but keeps making avoidable mistakes after learning. Decreasing ϵ can reduce that cost, provided exploration still visits the relevant state-action pairs often enough.

An exploration function instead combines value and visit count into an optimistic score $f(u, n)$. For example, give a high score R^+ while a pair has fewer than N_e visits, then use its estimated value. Replacing $\max_{a'} Q(s', a')$ by $\max_{a'} f(Q(s', a'), N(s', a'))$ in a target can propagate the incentive to reach unexplored states backward. The bonus is an exploration device, not a change to the actual reward objective.

Regret measures the expected reward lost relative to optimal behavior over the learning period. Two methods can converge to the same optimal policy and still have very different regret because they make different mistakes on the way.

11.2 Approximate Q-learning

A table cannot generalize to unvisited states and can be too large to store. Represent a state-action pair by features $f_i(s, a)$, such as distances to food or ghosts, and use a linear approximation

$$Q_w(s, a) = \sum_i w_i f_i(s, a).$$

Experience now changes a shared set of weights. Define the TD error

$$\delta = r + \gamma \max_{a'} Q_w(s', a') - Q_w(s, a), \qquad w_i \leftarrow w_i + \alpha \delta f_i(s, a).$$

This is a gradient step on squared prediction error when the sampled target is held fixed: the derivative of the prediction with respect to w_i is $f_i(s, a)$. A negative TD error lowers the prediction for the current pair and changes estimates for other pairs sharing its features. With one indicator feature per pair, the rule recovers tabular Q-learning.

Generalization saves data and memory, but states with the same features may have different values. Capacity limits can reduce overfitting, while an inadequate feature representation can prevent a good policy. The tabular convergence guarantee does not automatically carry over to approximation.

11.3 Policy search

Accurate numerical value estimates are a means to choosing good actions; their ordering can matter more than their absolute errors. Policy search directly adjusts policy parameters to improve expected return. A simple approach perturbs feature weights and compares returns from sampled episodes. Random outcomes require repeated trials, and testing each parameter separately becomes expensive in high dimension. Robotic locomotion and manipulation illustrate why generalization and learning effective policies are needed when enumerating all states is infeasible.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 11.

Lecture 12: Probability

12.1 Distributions, events, and conditioning

A random variable represents an uncertain quantity and has a domain of possible values. A discrete probability distribution assigns nonnegative mass summing to one. A joint distribution assigns a probability to every complete assignment of a collection of variables. With n variables of domain size d, its table has d^n entries.

An event is a set of assignments; its probability is the sum of their masses. To obtain a marginal, sum out variables not of interest: $P(x) = \sum_y P(x, y)$. For $P(e) > 0$, conditioning restricts attention to assignments satisfying the evidence and renormalizes:

$$P(x \mid e) = \frac{P(x, e)}{P(e)}, \qquad P(e) = \sum_x P(x, e).$$

A distribution such as $P(X \mid e)$ is a table over X; an expression with a specified lowercase value, $P(x \mid e)$, is a number. A family $P(X \mid Y)$ contains a normalized distribution for every value of Y.

For the joint masses $P(\text{hot}, \text{sun}) = 0.4$, $P(\text{hot}, \text{rain}) = 0.1$, $P(\text{cold}, \text{sun}) = 0.2$, and $P(\text{cold}, \text{rain}) = 0.3$, marginalizing gives $P(\text{cold}) = 0.5$. Selecting the cold rows and dividing by 0.5 gives $P(\text{sun} \mid \text{cold}) = 0.4$.

12.2 Product, chain, and Bayes rules

The definition of conditioning gives the product rule $P(x, y) = P(x \mid y)P(y)$. Repeated application yields the chain rule, valid without independence assumptions:

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}).$$

Writing $P(x, e)$ in both orders gives Bayes' rule:

$$P(x \mid e) = \frac{P(e \mid x)P(x)}{\sum_{x'} P(e \mid x')P(x')}.$$

The prior $P(x)$ is reweighted by the likelihood $P(e \mid x)$ to obtain the posterior. A likely observation under a hypothesis does not by itself make the hypothesis likely: its prior and competing explanations also matter. For a noisy location sensor, multiply each candidate location's prior by the probability of the observed reading at that location, then normalize.

12.3 Inference by enumeration

Partition variables into query Q, evidence $E = e$, and hidden variables H. Select the joint entries matching e, sum out H, and normalize:

$$P(Q \mid e) = \alpha \sum_{h} P(Q, h, e).$$

Here α is the reciprocal of the sum of the resulting entries, provided the evidence has positive probability. Enumeration is exact, but a full joint table needs exponential storage and generally exponential work. Structured models will express the joint using smaller local tables.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 12.

Lecture 13: Bayes Nets: Representation

13.1 Conditional independence

Two variables are independent when $P(x, y) = P(x)P(y)$ for all values. They are conditionally independent given Z, written $X \perp\!\!\!\perp Y \mid Z$, when, for all z of positive probability,

$$P(x, y \mid z) = P(x \mid z)P(y \mid z).$$

Equivalently, knowing Y adds no information about X once Z is known. Conditional independence need not imply unconditional independence. For example, two sensor readings can both depend on a ghost's location while being independent once the location is fixed.

13.2 Graph and local probabilities

A Bayesian network is a directed acyclic graph with one random variable per node and a conditional probability table (CPT) $P(X_i \mid \mathrm{Pa}(X_i))$ per node. It defines

$$P(x_1, \ldots, x_n) = \prod_i P(x_i \mid \mathrm{pa}(x_i)).$$

Choose a topological order. The modeling assumption replaces all earlier variables in the chain rule by the node's parents: the node is independent of earlier nonparents given its parents. The product is normalized because variables can be summed out in reverse topological order, each CPT summing to one for fixed parent values.

In the alarm network, burglary B and earthquake E influence alarm A; the alarm influences calls J and M:

$$P(B, E, A, J, M) = P(B)P(E)P(A \mid B, E)P(J \mid A)P(M \mid A).$$

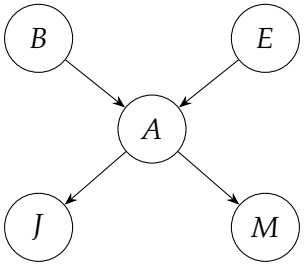


Figure 2: The alarm network's local dependencies. All variables are unobserved.

A network of independent coin flips has no edges and factors into individual priors. For n Boolean variables with at most k parents each, CPT storage is $O(n2^k)$ rather than $O(2^n)$ for a joint table. This saving depends on the graph's sparsity.

13.3 Representation and causality

A topology restricts which distributions can be represented by requiring certain conditional independences. A fully connected acyclic graph can express any joint distribution through the chain rule. Causal ordering often makes the model easier to specify, but arrows in a general Bayes net assert a factorization, not necessarily causation. A two-variable joint can be factored in either direction, with appropriately changed priors and CPTs. Observational association alone does not establish which variable causes the other.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 13.

Lecture 14: Bayes Nets: Independence

14.1 Three local configurations

Independence is a property of a distribution. A graph guarantees some independences for every choice of its CPTs; a particular numerical model can have additional independences.

In a chain $X \to Y \to Z$, observing the middle variable blocks the connection: $X \perp\!\!\!\perp Z \mid Y$. Without that observation, changes in X can change beliefs about Y and hence Z. A fork $X \leftarrow Y \to Z$ behaves similarly: the effects can be associated through a shared cause, but become independent given that cause, in the isolated three-node model.

A collider $X \to Y \leftarrow Z$ behaves oppositely. With independent root causes, summing out Y leaves $P(X)P(Z)$, so $X \perp\!\!\!\perp Z$. Observing Y can couple them: evidence for one explanation can reduce the need for the other. Observing a descendant of Y can also activate this connection. This is EXPLAINING AWAY.



Figure 3: Shading marks observation. The arrow pattern determines its effect on a path.

14.2 D-separation

To test whether X and Y are independent given evidence variables E, consider paths in the underlying undirected graph while retaining arrow directions to classify triples. A path is active exactly when:

- every noncollider on it is unobserved;
- every collider is observed or has an observed descendant.

One inactive triple blocks the entire path. If every path is blocked, X and Y are D-SEPARATED by E and $X \perp\!\!\!\perp Y \mid E$ in every distribution represented by the network.

An active path means independence is not guaranteed by the graph, not that dependence must occur for every CPT. Other paths must always be checked: blocking one route does not establish independence if another remains active. The rules depend on which nodes are observed, not on the numerical values of that evidence, for the graph's universal guarantees.

14.3 Modeling consequences

D-separation exposes assumptions made by selecting a topology. Adding edges can remove guaranteed independences and enlarge the represented family, but also enlarges CPTs and can make inference more expensive. The graph is a compact description of allowable distributions; numerical inference is a separate task requiring its CPTs.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 14.

Lecture 15: Bayes Nets: Inference

15.1 Factors and elimination

For query variables Q, hidden variables H, and evidence e,

$$P(Q \mid e) = \alpha \sum_{H} \prod_{i} P(X_i \mid \mathrm{Pa}(X_i))\big|_{E=e}.$$

A factor is a table over its unassigned variables; it need not be a normalized distribution. Selecting an observed value removes that dimension. Joining factors multiplies their entries over the union of their variables; summing out a variable adds entries that agree on all remaining variables.

Enumeration first constructs the joint and then marginalizes. Variable elimination interleaves these operations. To eliminate hidden variable H, collect every factor containing H, multiply them, sum out H, and return the resulting factor to the collection. Factors not containing H remain unchanged. After eliminating all hidden variables, multiply remaining factors and normalize over Q.

The justification is distributivity: factors independent of the summed variable can be pulled outside its sum. For the chain $R \to T \to L$,

$$P(L) = \sum_{t} P(L \mid t) \underbrace{\sum_{r} P(r)P(t \mid r)}_{f(t)}.$$

The inner result is stored once for each t, avoiding a full table over R, T, L. With $P(r) = 0.1$, $P(t \mid r) = 0.8$, and $P(t \mid \neg r) = 0.1$, $f(t) = 0.17$. If $P(l \mid t) = 0.3$ and $P(l \mid \neg t) = 0.1$, then $P(l) = 0.17(0.3) + 0.83(0.1) = 0.134$ and $P(\neg l) = 0.866$.

15.2 Evidence and normalization

Instantiate evidence in every factor containing an observed variable before elimination. Preserve the query variables. The final unnormalized table is $P(Q, e)$, whose sum is $P(e)$. Dividing by this sum gives the posterior. Normalizing an intermediate factor without tracking its scale can lose an evidence likelihood, so retain factors as unnormalized tables until the desired final normalization.

15.3 Ordering and complexity

A factor over k variables with domains of size d has d^k entries. Thus time and space depend critically on the largest intermediate factor, which depends on elimination order. Eliminating a highly connected variable early can join many otherwise separate variables into a large table. The answer is independent of order; the amount of computation is not.

There is no universally efficient ordering: general Bayes-net inference is NP-hard. One can encode a satisfiability problem so that a designated event has positive probability exactly when a satisfying assignment exists. Sparse structures such as polytrees admit efficient inference relative to their local table sizes. Cutset conditioning trades enumeration over selected variables for simpler remaining graphs, paralleling the structural methods for CSPs.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 15.

Lecture 16: Bayes Nets: Sampling

16.1 Prior and rejection sampling

To sample a finite distribution, partition $[0, 1)$ into intervals whose lengths equal its probabilities, draw a uniform number, and select its interval. Prior sampling applies this in topological order, drawing each node from its CPT given its sampled parents. The probability of a full sample is the product of these conditionals, exactly the network's joint.

Estimate an event by its frequency among samples. To estimate $P(Q \mid e)$, rejection sampling discards samples inconsistent with the evidence and tallies query values in the rest. Accepted samples have the desired conditional distribution. If $P(e)$ is small, only about $NP(e)$ of N prior samples survive; impossible evidence yields no posterior estimate.

16.2 Likelihood weighting

Likelihood weighting fixes evidence variables to their observed values and samples all other variables in topological order. Initialize $w = 1$. At an evidence node $E_i = e_i$, multiply w by $P(e_i \mid \text{pa}(E_i))$ instead of drawing its value. Return the complete assignment and its weight, and normalize weighted query counts:

$$\widehat{P}(q \mid e) = \frac{\sum_{j=1}^{N} w_j \mathbf{1}\{Q_j = q\}}{\sum_{j=1}^{N} w_j}.$$

Writing z for sampled variables, the proposal contains the non-evidence CPTs, while the weight contains the evidence CPTs:

$$q(z) = \prod_{i \notin E} P(z_i \mid \text{pa}_i), \qquad w(z) = \prod_{i \in E} P(e_i \mid \text{pa}_i), \qquad q(z)w(z) = P(z, e).$$

This identity explains consistency of the weighted estimate. Evidence affects downstream draws immediately, but does not guide upstream draws; a few samples can therefore dominate the total weight.

16.3 Gibbs sampling

Gibbs sampling keeps a complete assignment consistent with the evidence and repeatedly resamples one non-evidence variable given all the others. For node X,

$$P(x \mid \text{rest}) \;\propto\; P(x \mid \text{pa}(X)) \prod_{Y \text{ child of } X} P(y \mid \text{pa}(Y)).$$

All other CPTs cancel because they do not vary with x. Consequently the parents, children, and other parents of the children supply the needed local information. Both upstream and downstream observations can influence a draw.

Under conditions ensuring that this Markov chain explores the posterior's support and converges, long-run frequencies estimate the posterior. Successive samples are dependent, and slow mixing or deterministic constraints can make convergence difficult. More samples improve accuracy only insofar as they provide useful exploration of the distribution.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 16.

Lecture 17: Decision Networks and Value of Information

17.1 Decisions and expected utility

A decision network augments a Bayes net with decision nodes and a utility node. Chance nodes describe uncertainty, a decision node specifies an action, and utility depends on the action and relevant outcomes. For current evidence e, evaluate each action by

$$EU(a \mid e) = \sum_s P(s \mid e)U(a, s), \qquad MEU(e) = \max_a EU(a \mid e),$$

in the setting where the action does not change the chance distribution. More general action-dependent outcomes use the corresponding model for each action. Choose the maximizing action after inferring the utility parents.

For the umbrella example, utilities for leaving it are 100 in sun and 0 in rain; taking it gives 20 in sun and 70 in rain. With sun probability 0.7, leaving has value 70 and taking has value 35. A bad forecast that changes sun probability to 0.34 changes these to 34 and 53, making taking the umbrella preferable.

17.2 Value of information

Before observing a variable Z, its value is uncertain. The VALUE OF PERFECT INFORMATION about that variable is

$$VPI(Z \mid e) = \sum_z P(z \mid e) MEU(e, z) - MEU(e).$$

The maximum is taken separately after each possible observation: information is useful because the action can adapt. In the oil example, exactly one of two equally likely locations holds oil worth k. Choosing without a survey has value $k/2$; learning its location gives value k, so the information is worth $k/2$ in the same utility units.

VPI is nonnegative because the agent can ignore the observation and keep the old action. It can be zero even for a surprising observation if no action changes. Information values are not generally additive: two observations can overlap or become useful only together. Total value of observing a set is independent of its order, although the conditional value of the second observation depends on what is already known. Information acquisition is worthwhile when its expected benefit exceeds its cost in utility.

A noisy report is represented by observing a noisy sensor node, not by pretending to observe the underlying state. If the new variable is conditionally independent of the relevant uncertain utility parents given current evidence and every candidate action, it cannot improve the decision.

17.3 Partial observability

A PARTIALLY OBSERVABLE MDP adds an observation model to an MDP. The agent maintains a BELIEF STATE, a probability distribution over physical states. An action changes the physical state; an observation updates the belief. Planning can therefore be viewed as an MDP over beliefs, though the belief space is typically continuous and much larger.

In static Ghostbusters, beliefs are determined by accumulated sensor evidence. A truncated expectimax tree can compare acting immediately against sensing once and then acting. This produces a VPI-based sensing rule, while longer planning can account for sequences of information-gathering actions.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 17.

Lecture 18: Hidden Markov Models

18.1 Markov chains and stationary distributions

A first-order Markov chain represents a sequence of states with $P(X_{t+1} \mid X_{1:t}) = P(X_{t+1} \mid X_t)$. A stationary transition model uses the same conditional table at every time. Starting from a distribution b_t, prediction pushes probability through the transitions:

$$b_{t+1}(x') = \sum_x P(x' \mid x) b_t(x).$$

A stationary distribution satisfies $\pi(x') = \sum_x P(x' \mid x)\pi(x)$. It need not be the limit from every initial distribution for an arbitrary chain; convergence requires additional conditions. In the weather example, $P(\text{sun} \mid \text{sun}) = 0.9$ and $P(\text{sun} \mid \text{rain}) = 0.3$ give $\pi(\text{sun}) = 0.9\pi(\text{sun}) + 0.3\pi(\text{rain})$, hence $\pi(\text{sun}) = 3/4$ and $\pi(\text{rain}) = 1/4$.

PageRank uses a random walk on pages with occasional random jumps, assigning larger stationary mass to more reachable pages. Gibbs sampling similarly constructs a chain over complete assignments whose stationary distribution is the desired posterior.

18.2 Hidden states and observations

A hidden Markov model (HMM) adds observations E_t emitted from hidden states X_t. It specifies an initial state distribution, transitions $P(X_t \mid X_{t-1})$, and emissions $P(E_t \mid X_t)$:

$$P(x_{1:T}, e_{1:T}) = P(x_1)P(e_1 \mid x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})P(e_t \mid x_t).$$

The hidden process is Markov, and the current observation is independent of the other variables given the current state. Observations are not generally independent without conditioning on states.

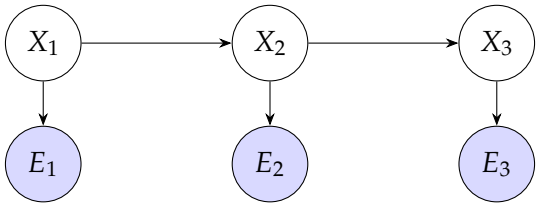


Figure 4: Hidden dynamics and observed emissions in an HMM.

18.3 Filtering and the forward algorithm

Filtering maintains $b_t(x) = P(X_t = x \mid e_{1:t})$. Prediction and observation updates are

$$\bar{b}_{t+1}(x') = \sum_x P(x' \mid x) b_t(x), \qquad b_{t+1}(x') = \alpha P(e_{t+1} \mid x') \bar{b}_{t+1}(x').$$

The first equation follows by marginalizing the previous state and using the Markov property. The second is Bayes' rule and emission independence. Initialize by weighting the initial prior with $P(e_1 \mid x_1)$. Predictions often spread belief; informative observations often concentrate it, although an individual observation need not reduce entropy.

The unnormalized forward algorithm instead carries $f_t(x) = P(x_t, e_{1:t})$:

$$f_{t+1}(x') = P(e_{t+1} \mid x') \sum_x P(x' \mid x) f_t(x).$$

Normalize to recover the filtering distribution; scaling each step avoids numerical underflow. With $|S|$ states, a dense update costs $O(|S|^2)$ and stores only the current vector. In robot localization, states are positions and observations are noisy sensor readings; observations disambiguate positions that the motion model alone cannot distinguish.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 18.

Lecture 19: Particle Filtering and Applications of HMMs

19.1 Particles as a belief representation

Exact filtering is impractical when the state space is enormous or continuous. A particle filter represents a belief with N sampled states $x^{(1)}, \ldots, x^{(N)}$ rather than a table over all possible states. In a discrete space, the fraction equal to x approximates its probability. Store the particles themselves; a table over every possible state defeats the space saving.

One filtering step has three operations:

1. Predict each particle independently by drawing $\tilde{x}^{(i)} \sim P(X_{t+1} \mid x^{(i)})$.
2. Weight the predicted particles by the fixed observation: $w_i = P(e_{t+1} \mid \tilde{x}^{(i)})$.
3. Draw N new particles with replacement from the predicted particles, with selection probability $w_i / \sum_j w_j$.

The resulting particles again have equal weights. Prediction approximates the transition sum; weighting and resampling approximate the observation update. Resampling replicates plausible states and removes implausible ones, but introduces sampling noise. If all weights are zero, normalization is undefined and the representation must be recovered rather than silently continuing. More particles help, but difficult beliefs may require many.

With efficient transition draws, likelihood evaluation, and resampling, work is linear in N per step. In localization the map is known and the robot's pose is uncertain. In simultaneous localization and mapping (SLAM), both pose and map are unknown, so the state is substantially larger.

19.2 Dynamic Bayes nets

A dynamic Bayes net repeats a structured collection of variables at each time, with dependencies within a slice and from the preceding slice. It generalizes an HMM whose single state variable encodes the whole slice.

Exact inference unrolls the network and eliminates old variables, retaining factors needed for the present. These factors may become large. For a particle filter, each particle must be a complete joint assignment to the slice, such as both ghosts' positions. Sample its successor from the joint transition model and weight it by the observation likelihood. When emissions are conditionally independent, multiply their likelihood factors. Sampling each tracked entity independently would lose dependencies represented by the joint model.

19.3 Most likely state sequences

Filtering asks for a marginal distribution at the current time. A MOST LIKELY EXPLANATION instead asks for the single sequence $\arg\max_{x_{1:T}} P(x_{1:T} \mid e_{1:T})$. The evidence normalizer does not depend on the sequence, so maximize the joint.

The VITERBI ALGORITHM replaces forward sums over paths by maxima:

$$v_1(x) = P(x)P(e_1 \mid x), \qquad v_{t+1}(x') = P(e_{t+1} \mid x') \max_x [P(x' \mid x) v_t(x)].$$

Store the maximizing predecessor for each state and time. Choose the best final state, then trace back these pointers. A state trellis makes the distinction clear: forward inference sums the weights of all paths reaching a node, whereas Viterbi retains only the best path. Choosing the most likely state separately at each time need not give the most likely valid sequence.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 19.

Lecture 20: Machine Learning: Naive Bayes

20.1 Classification and the model

Supervised classification learns from labeled inputs (x, y) to predict labels for new inputs. Features encode properties such as word occurrences, sender identity, or image pixels. A naive Bayes classifier assumes that features are conditionally independent given the label:

$$P(y, f_1, \ldots, f_n) = P(y) \prod_i P(f_i \mid y), \qquad \hat{y} = \arg\max_y P(y) \prod_i P(f_i \mid y).$$

Normalize the class scores for a posterior distribution. Conditional independence is a modeling assumption, not a claim that features are unconditionally independent. The model has only linearly many local feature tables rather than an exponential table over all features.

For text, a bag-of-words model makes word positions conditionally independent and identically distributed given the class, tying $P(W_i \mid Y)$ across positions. The score is insensitive to order. Compute log scores, $\log P(y) + \sum_i \log P(w_i \mid y)$, to avoid underflow.

20.2 Parameter estimation and smoothing

Maximum likelihood estimates categorical probabilities by relative frequencies. For a feature value v in class y,

$$\widehat{P}(F_i = v \mid y) = \frac{N(F_i = v, y)}{N(y)}.$$

An unseen event then receives probability zero, eliminating a class no matter how strong the other evidence is. Laplace smoothing adds $k > 0$ pseudo-counts to each of the d_i possible feature values:

$$\widehat{P}(F_i = v \mid y) = \frac{N(F_i = v, y) + k}{N(y) + kd_i}.$$

Smooth each conditioning context separately. In a word-position model, count tokens within each class and use the vocabulary size in the denominator. Larger k pulls toward a uniform distribution. Linear interpolation is an alternative: mix the empirical conditional with an overall empirical distribution, reducing reliance on sparse conditional counts.

20.3 Generalization and evaluation

Training learns parameters. A separate held-out set selects hyperparameters such as smoothing strength; the test set estimates final performance and must not guide repeated tuning. Overfitting occurs when training peculiarities or noise are fitted at the expense of future prediction. More data, simpler representations, and regularization can help.

Compare against a baseline such as always predicting the most frequent training label. High accuracy on an imbalanced problem can conceal a weak classifier. Inspecting errors can reveal missing features, such as whether the sender is known or whether a displayed link matches its destination.

A probabilistic classifier's top posterior is a confidence estimate, but the model can be wrong about that confidence. Calibration asks whether predictions made with a given confidence are correct at the corresponding rate; a high numerical posterior alone is no guarantee of reliability.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 20.

Lecture 21: Machine Learning: Perceptrons and Logistic Regression

21.1 Linear scores and perceptron updates

A feature vector $f(x)$ maps an input to numbers. A linear classifier scores it by $z = w^\top f(x)$ and predicts the sign. A constant feature represents a bias, allowing the separating hyperplane to move away from the origin.

The binary PERCEPTRON starts with zero weights. For each example with gold label $y^* \in \{-1, +1\}$, predict using the current score. On a mistake, update $w \leftarrow w + y^* f(x)$; otherwise leave it unchanged. The update raises the gold label's signed score on that example.

For multiple classes, keep a vector w_y for each class and predict $\hat{y} = \arg\max_y w_y^\top f(x)$. On a mistake,

$$w_{y^*} \leftarrow w_{y^*} + f(x), \qquad w_{\hat{y}} \leftarrow w_{\hat{y}} - f(x).$$

Only the correct and incorrectly predicted class vectors change. The relative score of the correct class increases on the mistaken input.

If the binary training set is linearly separable, perceptron training converges after finitely many mistakes; the margin affects the mistake bound. Nonseparable data can make weights keep changing. Averaging successive weight vectors can stabilize predictions. A separating boundary need not generalize well, and repeated passes can overfit; held-out performance helps choose when to stop.

21.2 Logistic probabilities

Perceptron scores are not probabilities. Binary LOGISTIC REGRESSION uses the sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad P(y = +1 \mid x; w) = \sigma(w^\top f(x)), \quad P(y = -1 \mid x; w) = 1 - \sigma(w^\top f(x)).$$

The sign still determines the most likely class, but the score's magnitude controls confidence. Learn weights by maximizing conditional log likelihood:

$$\ell(w) = \sum_i \log P(y^{(i)} \mid x^{(i)}; w).$$

This objective uses degrees of confidence rather than changing weights only on mistakes. It can learn probabilistic predictions even when no linear boundary perfectly separates the labels.

21.3 Multiclass logistic regression

For class scores $z_y = w_y^\top f(x)$, SOFTMAX gives

$$P(y \mid x; w) = \frac{e^{z_y}}{\sum_{y'} e^{z_{y'}}}.$$

The largest score remains the most probable label, and the probabilities sum to one. Optimize the sum of the gold labels' log probabilities on training data. All class scores enter the normalizer, so increasing one class's score changes the probabilities of the others. Subtracting a common constant from the scores preserves these probabilities and permits stable computation.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 21.

Lecture 22: Machine Learning: Optimization and Neural Networks

22.1 Gradient optimization

To maximize a differentiable objective $g(w)$, gradient ascent uses $w \leftarrow w + \alpha \nabla g(w)$. The first-order approximation $g(w+\Delta) \approx g(w) + \nabla g(w)^\top \Delta$ explains the direction: among steps with fixed Euclidean length, the gradient has greatest predicted increase. The learning rate α controls distance; an excessively large step can overshoot, while a small step can make progress slow.

For conditional log likelihood $\ell(w) = \sum_i \log P(y^{(i)} \mid x^{(i)}; w)$, batch ascent sums gradients over the entire training set before updating. Stochastic ascent updates from a randomly selected example, and mini-batch ascent uses a small random subset, allowing parallel computation with more frequent updates than a full batch. Their gradient scaling and learning rate must be used consistently.

For a binary label $t \in \{0, 1\}$ and $p = \sigma(w^\top f)$, differentiating $t \log p + (1-t)\log(1-p)$ gives $(t-p)f$. This displays the training signal: raise scores where the observed label exceeds the predicted probability.

22.2 Learning representations

Logistic regression applies a learned linear score to fixed features. A NEURAL NETWORK learns intermediate features as well:

$$h^{(0)} = x, \qquad h^{(k)} = g(W^{(k)} h^{(k-1)} + b^{(k)}), \qquad P(y \mid x) = \text{softmax}(W^{\text{out}} h^{(L)} + b^{\text{out}})_y.$$

Nonlinear activations such as sigmoid, hyperbolic tangent, or $\text{ReLU}(z) = \max(0, z)$ permit nonlinear input-output functions. Without nonlinearities, composing affine layers would remain affine.

Training again maximizes conditional log likelihood, now over all layers' weights. Large networks can fit complex structure and can also memorize training data. Early stopping uses held-out performance to limit overfitting; training accuracy alone does not identify the best stopping point.

22.3 Approximation and automatic differentiation

With suitable nonlinear activations, a network with one sufficiently wide hidden layer can approximate continuous functions on a compact domain to arbitrary accuracy. This is an expressiveness result; it does not guarantee that training finds the weights, that the network is small, or that it generalizes from finite data.

A network is a composition of simple operations, so the chain rule computes its derivatives. AUTOMATIC DIFFERENTIATION records intermediate values on a forward pass and propagates derivative information backward. This BACKPROPAGATION computes gradients with respect to many parameters at a cost comparable in order to evaluating the network, rather than separately perturbing every weight. Learned features support tasks such as object recognition, speech recognition, translation, and captioning.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 22.

Lecture 23: Machine Learning: Neural Networks II and Decision Trees

23.1 Induction, consistency, and simplicity

Learning chooses a hypothesis h from a hypothesis space H using examples of an unknown target function. Classification predicts discrete labels; regression predicts real values. A hypothesis consistent with all observed examples can still be wrong elsewhere. Expressive models, including neural networks, can represent more targets but also fit more accidental patterns.

The tradeoff between consistency and simplicity can be controlled by reducing the hypothesis space, selecting features, adding modeling assumptions, or regularizing selection within a large space. These choices balance underfitting against sensitivity to the sample, often described as bias and variance. Held-out performance evaluates that balance.

23.2 Decision trees and expressiveness

A decision tree tests an attribute at each internal node, follows the matching branch, and predicts at a leaf. Each root-to-leaf path represents a conjunction of tests. A feature can have different effects in different branches, automatically capturing interactions that a linear classifier would need explicit conjunction features to represent.

For finitely many discrete attributes, a sufficiently large tree can represent any function by distinguishing complete assignments. With n Boolean attributes, there are 2^{2^n} Boolean functions, so unrestricted expressiveness is enormous. The practical aim is a small useful tree rather than a table memorizing every assignment.

23.3 Information gain and recursive learning

Grow a tree recursively by choosing an informative attribute, splitting examples by its values, and repeating within each subset. A pure subset can become a leaf. When no useful split remains, use a label based on the examples reaching the leaf.

For label proportions $p_1, \ldots, p_k$, ENTROPY is

$$H(Y) = -\sum_j p_j \log_2 p_j, \qquad 0 \log_2 0 := 0.$$

It measures average information in the label: a fair binary label has one bit, while a certain label has zero. The INFORMATION GAIN of attribute A is

$$IG(A) = H(Y) - \sum_a \frac{N_a}{N} H(Y \mid A = a).$$

Use subset frequencies N_a/N to weight the post-split entropies. Choose the largest gain and recurse. This greedy rule need not find the globally best tree: useful interactions can be invisible at an individual split.

23.4 Overfitting and pruning

Small subsets can appear pure by chance. Continuing to split them can improve training accuracy while harming generalization. One approach builds a tree and prunes bottom-up, removing splits whose apparent predictive relationship is not sufficiently supported, for example under a chi-squared significance criterion. Later structure can redeem a split that looks uninformative by itself, as with XOR, so premature stopping can miss interactions.

A pruning threshold is a regularization hyperparameter chosen on held-out data. A maximum-depth restriction instead directly limits the hypothesis space. Smaller trees can underfit, while larger trees can fit noise; the test set is reserved for evaluating the final choice.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 23.

Lecture 24: Advanced Applications: Robotics, Vision, and Control

24.1 Combining search and learning

Large applications combine the course's methods. In Go, exhaustive minimax is infeasible because of branching and depth. A policy network narrows plausible moves; a value network estimates positions without reaching the end. Supervised learning from expert play, reinforcement learning through self-play, and Monte Carlo rollouts contribute complementary information.

24.2 Helicopter estimation and control

A helicopter's state includes position, orientation, and their rates of change. Cameras and inertial sensors provide noisy measurements; a dynamics model predicts motion under control inputs:

$$s_{t+1} = f(s_t, a_t) + w_t.$$

Tracking combines prediction with observation updates, as in an HMM. Control chooses inputs affecting the main rotor and tail rotor. A hovering objective penalizes deviation from a target pose, but complex maneuvers also require specifying a useful target trajectory.

Demonstrations can be observations of an underlying desired trajectory. Their speeds differ, so directly averaging equal time indices blurs the maneuver. Probabilistic alignment combines a dynamics model with temporal alignment and state estimation to recover a cleaner latent trajectory. A controller can then track the learned maneuver. This connects imitation, sequence inference, and control rather than treating demonstrations as independent input-output examples.

24.3 Locomotion and rewards

Continuous-control policies can map joint angles and velocities through a neural network to joint torques. Learning in simulation permits repeated trials, while the representation must support coordinated motion. Hierarchical control separates a high-level choice, such as where to place a foot, from low-level motor commands that execute it.

For terrain traversal, a reward of the form $R(s) = w^\top f(s)$ can express preferences over terrain properties. Demonstrated paths help infer reward weights; planning with those weights then chooses paths across a new height map. Learning the objective can matter as much as optimizing a fixed one.

24.4 Autonomous driving and manipulation

An autonomous vehicle combines positioning, perception, planning, and feedback steering. Laser height differences can flag obstacles, but raw measurements contain errors. A temporal probabilistic model combines uncertain position and repeated sensor observations, reducing spurious detections relative to isolated threshold tests. Camera-based perception can learn from other sensors' reliable labels, an example of self-supervision. Neural semantic segmentation assigns a class to each image pixel.

Robotic laundry, object manipulation, and learning several physical skills add high-dimensional perception, contact dynamics, and generalization to the control problem. Learned visual and control representations connect observations to action, while interaction supplies data unavailable from static labeled images alone. These examples show why sensing uncertainty, reward design, computation, and learning must be addressed together.

Source. Dan Klein, Pieter Abbeel, and Anca Dragan, Introduction to Artificial Intelligence, Fall 2018, Lecture 24.

Lecture 25: Advanced Topics and Final Contest

25.1 Cooperative game strategies

Cooperative Pac-Man requires an agent to collect food while coordinating with another agent and responding to ghosts. The contest strategies illustrate several ways to combine the course's tools.

A feature-based reflex policy can weigh distance to food, teammate separation, and ghost danger. Seeking food farther from a nearby teammate reduces duplicated effort. Approximate Q-learning can instead learn weights for distance and score features, including a term that discourages deadlock.

A reward-density strategy first scores areas by food concentration, lowers scores where a teammate is likely to go, discounts distant targets, and penalizes nearby ghosts. Particle filtering maintains beliefs about teammate positions. A* selects routes to attractive areas when safe, while minimax handles immediate ghost danger. The central lesson is that estimation, search, local evaluation, and learning can serve different parts of one policy.

25.2 Speech as sequential inference

Speech recognition maps an acoustic waveform to words. Short-time spectral analysis describes energy across frequencies. Vocal-cord vibration supplies harmonics; the vocal tract amplifies selected frequency bands according to its shape. Different speakers, sounds, speaking rates, and transitions between sounds make matching waveforms directly unreliable.

An HMM uses acoustic feature vectors as observations and a PRONUNCIATION CURSOR as its hidden state: a position within a word's sequence of sounds. Emissions model how likely an acoustic vector is for that sound. Transitions advance within words and between words, allowing variation in duration and pronunciation.

A language model scores transitions between words. A bigram uses $P(w_t \mid w_{t-1})$; a trigram uses $P(w_t \mid w_{t-2}, w_{t-1})$. More context captures additional correlations but requires more data or parameter sharing. For example, the probability of "door" after "close the" can be far larger than after "the" alone.

Decoding finds the most likely state sequence given the acoustics, then reads the words from that sequence. This is a Viterbi query rather than a sequence of independent classifications:

$$\hat{x}_{1:T} = \arg\max_{x_{1:T}} P(x_1)P(e_1 \mid x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})P(e_t \mid x_t).$$

Acoustic and language models contribute different evidence. Neural networks can parameterize these models and learn richer representations than fixed local tables. Robust recognition at scale still differs from full contextual understanding of language.

25.3 Connecting inference to action

Physical Pac-Man demonstrations and a robot crawler connect simulation to sensing and actuation. A policy that works in a model must receive useful observations and issue executable actions in the physical system. The same division of work recurs throughout AI: represent uncertainty, estimate the state, compare consequences under an objective, and learn missing models or representations from experience.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 25.